

Q3

● Graded

Student

Steven Chen

Total Points

35.55 / 37 pts

Question 1

(no title)

0.75 / 1 pt

+ 1 pt `Flower<Bee> flower = new Tulip();`

✓ + 0.75 pts Minor errors

+ 0.5 pts Major errors

+ 0 pts Incorrect

Question 2

(no title)

1.8 / 2 pts

+ 2 pts a, d, e, f, h

✓ + 1.8 pts 1 incorrect

+ 1.6 pts 2 incorrect

+ 1.4 pts 3 incorrect

+ 0 pts All incorrect

Question 3

(no title)

8 / 8 pts

✓ + 8 pts `O(1) O(1)`
`O(1) O(n)`
`O(n) O(1)`
`O(n) O(n)`

+ 7 pts 1 incorrect

+ 6 pts 2 incorrect

+ 0 pts All incorrect

Question 4**(no title)****2 / 2 pts**

✓ **+ 2 pts** The worst case is if we need to resize the array, meaning we have to copy n elements to a new array. This takes $O(n)$ time.

Another worst case is if we insert at the head and have to shift the n subsequent elements down an index, which is also $O(n)$.

Needing to do both in succession would still be $O(n)$.

+ 0 pts Incorrect

Question 5**(no title)****2 / 2 pts**

✓ **+ 2 pts** The worst case would be if i is $n + 1$, meaning we have to traverse the LinkedList from index 0 (head) to n . This takes $O(n)$ time.

+ 0 pts Incorrect

Question 6**(no title)****1 / 1 pt**

✓ **+ 1 pt** 7

+ 0 pts Incorrect

Question 7**(no title)****1 / 1 pt**

✓ **+ 1 pt** 5

+ 0 pts Incorrect

Question 8**(no title)****1 / 1 pt**

✓ **+ 1 pt** d

+ 0 pts Incorrect

Question 9**(no title)****2 / 2 pts**

✓ **+ 2 pts** b, c, g, h

+ 1.75 pts 1 incorrect

+ 1.5 pts 2 incorrect

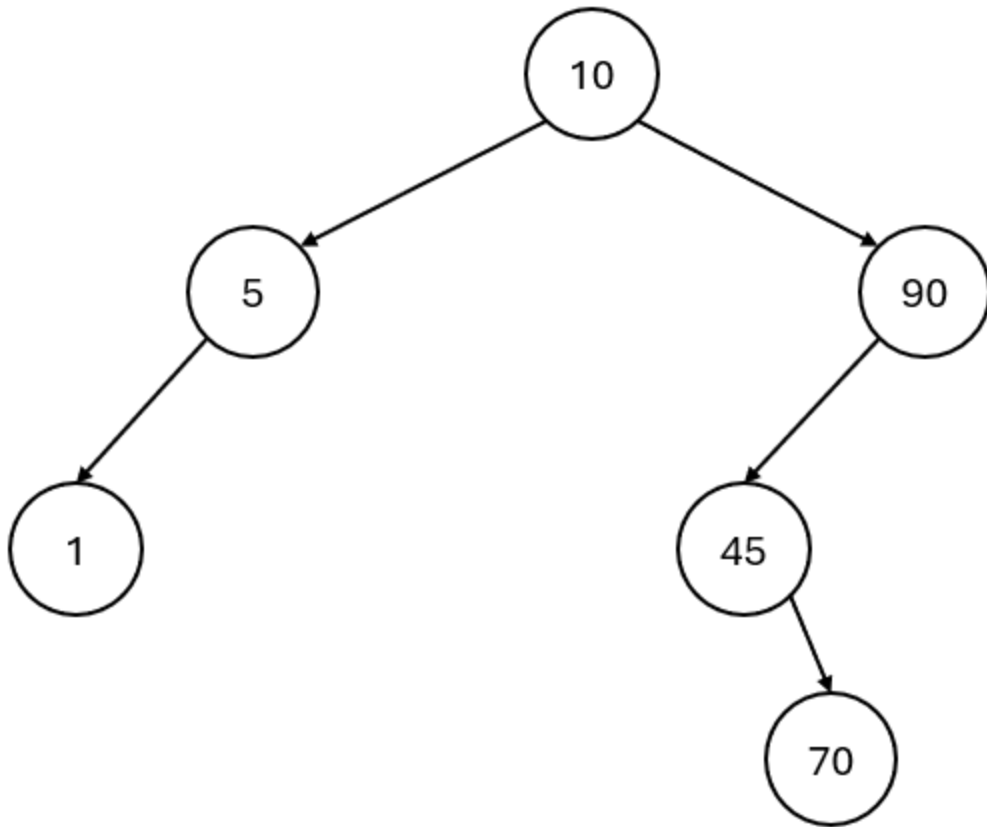
+ 0 pts All incorrect

Question 10

(no title)

3 / 3 pts

✓ + 3 pts



+ 0 pts Incorrect

Question 11

(no title)

2 / 2 pts

✓ + 2 pts 1, 5, 10, 45, 70, 90

+ 1.667 pts 1 incorrect

+ 0 pts Incorrect

Question 12

(no title)

2 / 2 pts

✓ + 2 pts 1, 5, 70, 45, 90, 10

+ 0 pts Incorrect

Question 13

(no title)

2 / 2 pts

✓ + 2 pts 10, 5, 90, 1, 45, 70

+ 0 pts Incorrect

Question 14

(no title)

1 / 1 pt

✓ + 1 pt $(i-1)/2$

+ 0.5 pts Minor errors

+ 0 pts Incorrect

Question 15

(no title)

1 / 1 pt

✓ + 1 pt $i * 2 + 2$

+ 0.5 pts Minor errors

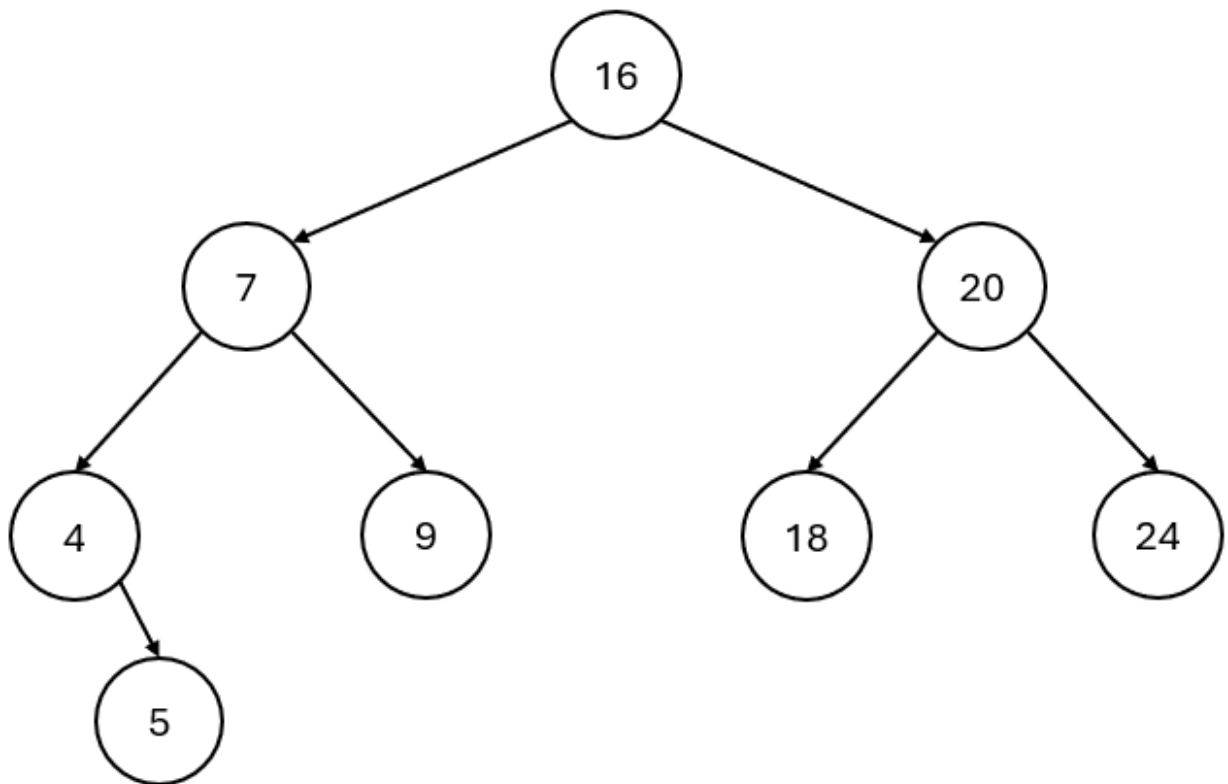
+ 0 pts Incorrect

Question 16

(no title)

3 / 3 pts

✓ + 3 pts



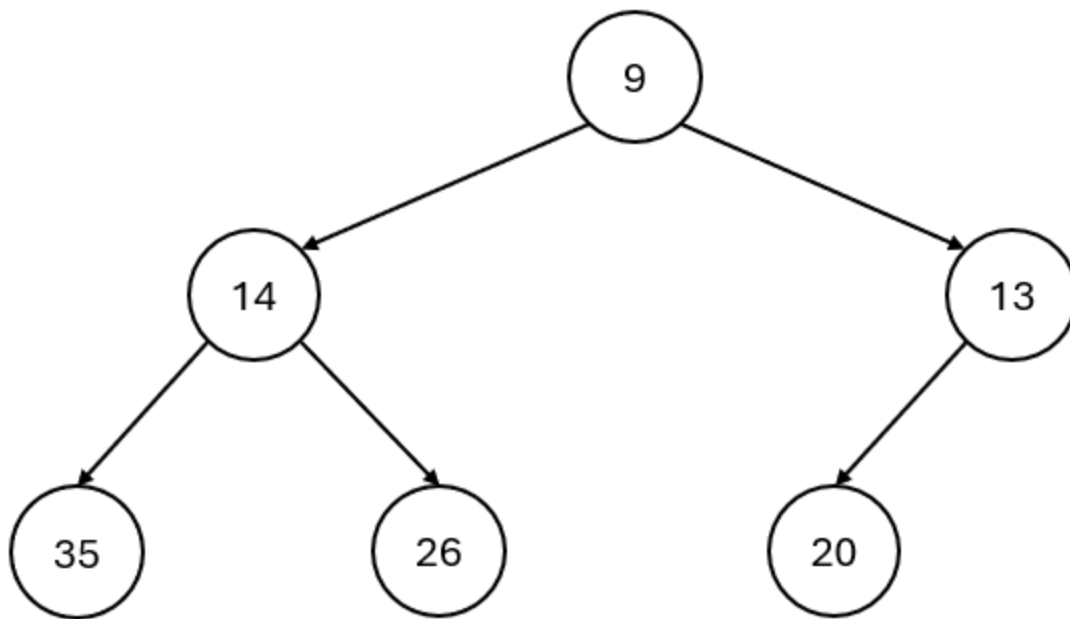
+ 2.75 pts Replaced 10 with the min from the left subtree instead of the max from the right subtree

+ 2.5 pts Minor errors

+ 1 pt Major errors

+ 0 pts Incorrect

+ 3 pts



+ 2.75 pts Enqueued 20 on the right instead of the left

✓ + 2 pts Rebalancing error

+ 1.5 pts Did not rebalance

+ 0 pts Incorrect

COMP 210 Q3

Name: Steven Chen

Consider the following code.

```
1 public interface Flower<T> {  
2     // body omitted  
3 }
```

```
1 public class Tulip implements Flower<Bee> {  
2     // body omitted  
3 }
```

1. (1 point) Declare a Flower object instantiated as a Tulip.

```
Flower<Bee> x = new Tulip<>;
```

2. (2 points) Which of the following are ADTs? Select all that apply.

- ☒ a. List
- b. ArrayList
- c. LinkedList
- ☒ d. Stack
- ☒ e. Queue
- ☒ f. Binary Tree
- g. Binary Search Tree
- ☒ h. Priority Queue
- i. Binary Min Tree
- ☒ j. Binary Heap

3. (8 points) Fill out the worst case time complexities of each method.

Method	ArrayList	LinkedList (singly-linked, no tail)
get(0)	$O(1)$	$O(1)$
get(i)	$O(1)$	$O(n)$
add(0)	$O(n)$	$O(1)$
add(i)	$O(n)$	$O(n)$

4. (2 points) Briefly explain your answer for the worst case time complexity of add(i) for an ArrayList.

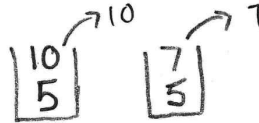
The worst case is $O(n)$ because you might add to a full array which causes a new array to be made and copied over.

5. (2 points) Briefly explain your answer for the worst case time complexity of add(i) for a LinkedList.

You may have to go through the entire linked list to get to i and then you could modify the pointers

Consider the following series of operations using an instance of the `StackImpl` class.

```
1 stack.push(5);  
2 stack.push(10);  
3 stack.pop();  
4 stack.push(7);  
5 stack.pop();
```



6. (1 point) What element is returned as the result of line 5?

7

7. (1 point) What are the contents of the stack after line 5?

5

8. (1 point) What is the best implementation of a Queue?

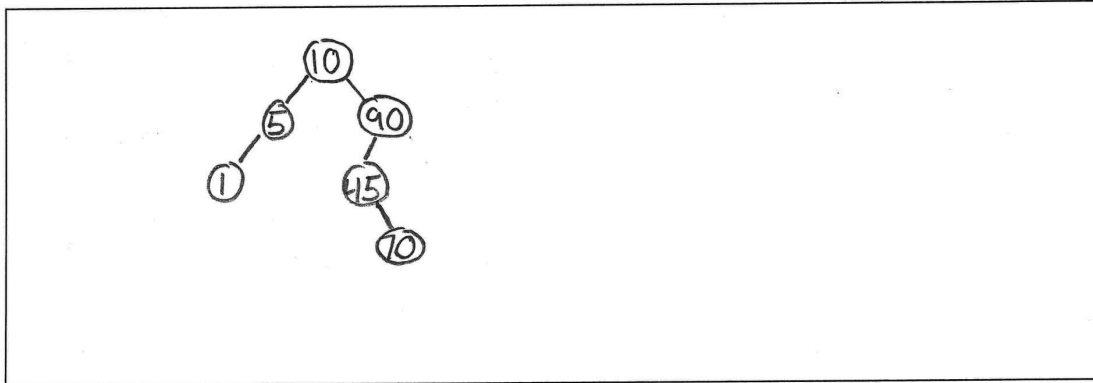
- a. ArrayList that enqueues at the head and dequeues at the tail
- b. ArrayList that enqueues at the tail and dequeues at the head
- c. LinkedList that enqueues at the head and dequeues at the tail
- ☒ d. LinkedList that enqueues at the tail and dequeues at the head

9. (2 points) Which of the following are invariants of a BST? Select all that apply.

- a. Root value is smaller than all values in the left subtree
- ☒ b. Root value is smaller than all values in the right subtree
- ☒ c. Root value is larger than all values in the left subtree
- d. Root value is larger than all values in the right subtree
- e. The left child is non-empty
- f. The right child is non-empty
- ☒ g. The left child is a BST
- ☒ h. The right child is a BST

10. (3 points) Insert the following values into an empty BST and draw the resulting tree:

10, 90, 5, 45, 1, 70



11. (2 points) What is the in-order traversal of your tree?

1, 5, 10, 45, 70, 90

12. (2 points) What is the post-order traversal of your tree?

1, 5, 70, 45, 90, 10

13. (2 points) What is the breadth-first traversal of your tree?

10, 5, 90, 1, 45, 70

Suppose I have a binary heap node at index i .

$$\text{parent} \cdot 2 + 1 = \text{child}$$

14. (1 point) What is the index of its parent?

$$\lfloor (i-1)/2 \rfloor$$

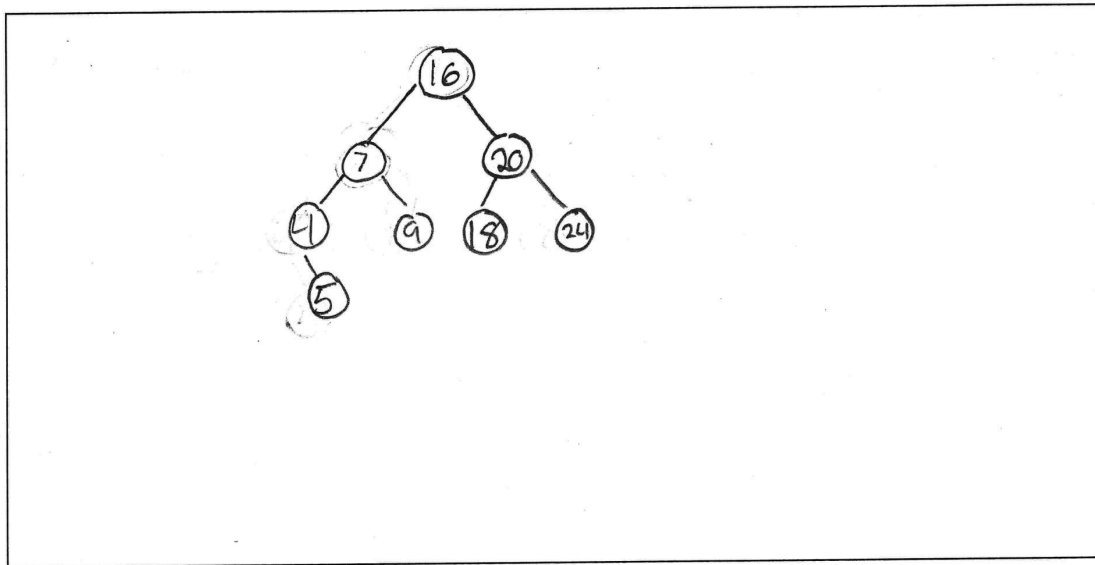
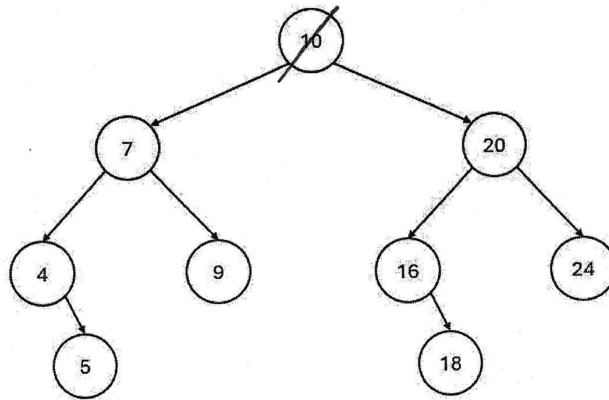
15. (1 point) What is the index of its right child?

$$(i \cdot 2) + 2$$

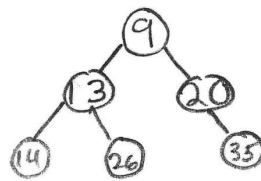
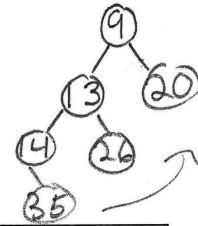
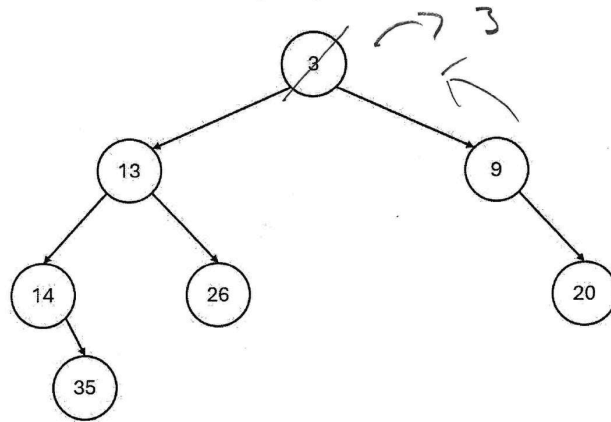
$$i = 3$$



16. (3 points) Draw the result of calling `remove(10)` on the following BST.



17. (3 points) Draw the result of calling dequeue() on the following BMT.



🎉🎉 Yippee! You did great!! 🎉🎉

